

Question 1.

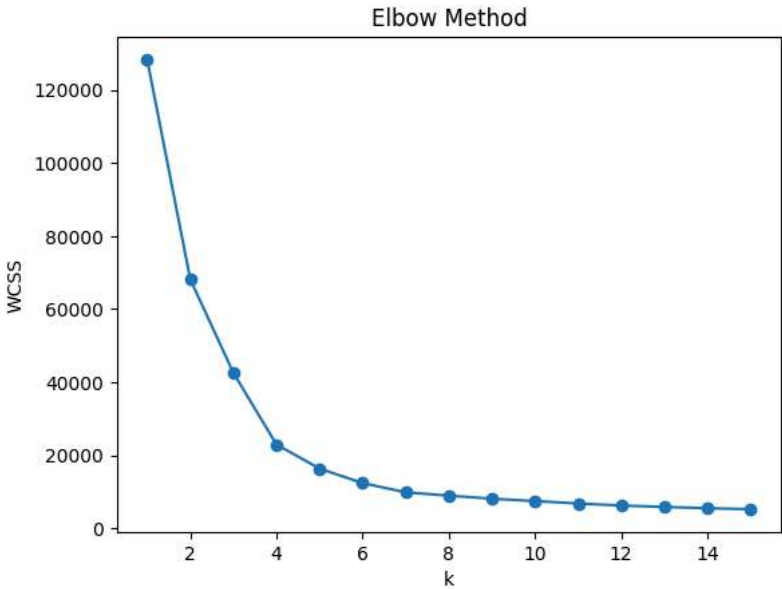
```
1 from google.colab import files
2 uploaded = files.upload()
3 for filename in uploaded.keys():
4     print(f'User uploaded file "{filename}" with length {len(uploaded[filename])} bytes')
```

Choose files A3_P1_7b2...3544adc.csv

A3_P1_7b27d5cc-755e-4f78-bab1-6c2373544adc.csv(text/csv) - 60523 bytes, last modified: 10/03/2026 - 100% done
Saving A3_P1_7b27d5cc-755e-4f78-bab1-6c2373544adc.csv to A3_P1_7b27d5cc-755e-4f78-bab1-6c2373544adc.csv
User uploaded file "A3_P1_7b27d5cc-755e-4f78-bab1-6c2373544adc.csv" with length 60523 bytes

Elbow method

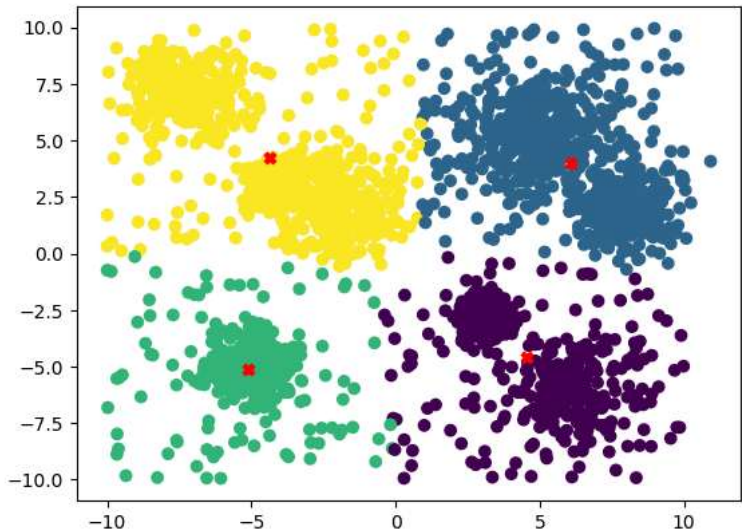
```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4 X = pd.read_csv('A3_P1_7b27d5cc-755e-4f78-bab1-6c2373544adc.csv').values
5
6 wcss = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(X).inertia_ for k in range(1,16)]
7 plt.plot(range(1,16), wcss, marker='o')
8 plt.xlabel("k"); plt.ylabel("WCSS"); plt.title("Elbow Method"); plt.show()
```



K-means clustering

```
1 n_clusters=4
2 print("Optimal number of cluster = ",n_clusters,"\n")
3 kmeans = KMeans(n_clusters, random_state=42, n_init=10)
4 labels = kmeans.fit_predict(X)
5
6 plt.scatter(X[:,0], X[:,1], c=labels)
7 plt.scatter(kmeans.cluster_centers_[0,0], kmeans.cluster_centers_[0,1], c='red', marker='x')
8 plt.show()
```

Optimal number of cluster = 4



```
1 from google.colab import files
2
3 uploaded = files.upload()
4
5 for filename in uploaded.keys():
6     print(f'User uploaded file "{filename}" with length {len(uploaded[filename])} bytes')
```

Choose files A3_P2_170...f772880.csv

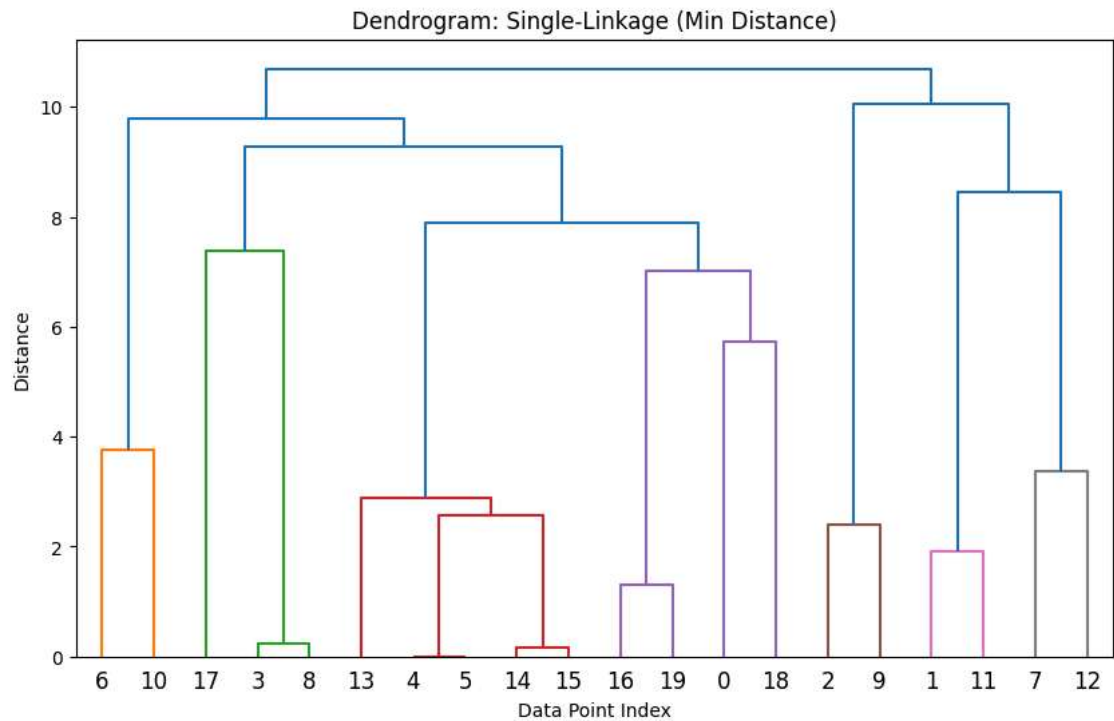
A3_P2_170cd1d8-a072-4f67-ba36-10f26f772880.csv(text/csv) - 379 bytes, last modified: 10/03/2026 - 100% done

Saving A3_P2_170cd1d8-a072-4f67-ba36-10f26f772880.csv to A3_P2_170cd1d8-a072-4f67-ba36-10f26f772880.csv

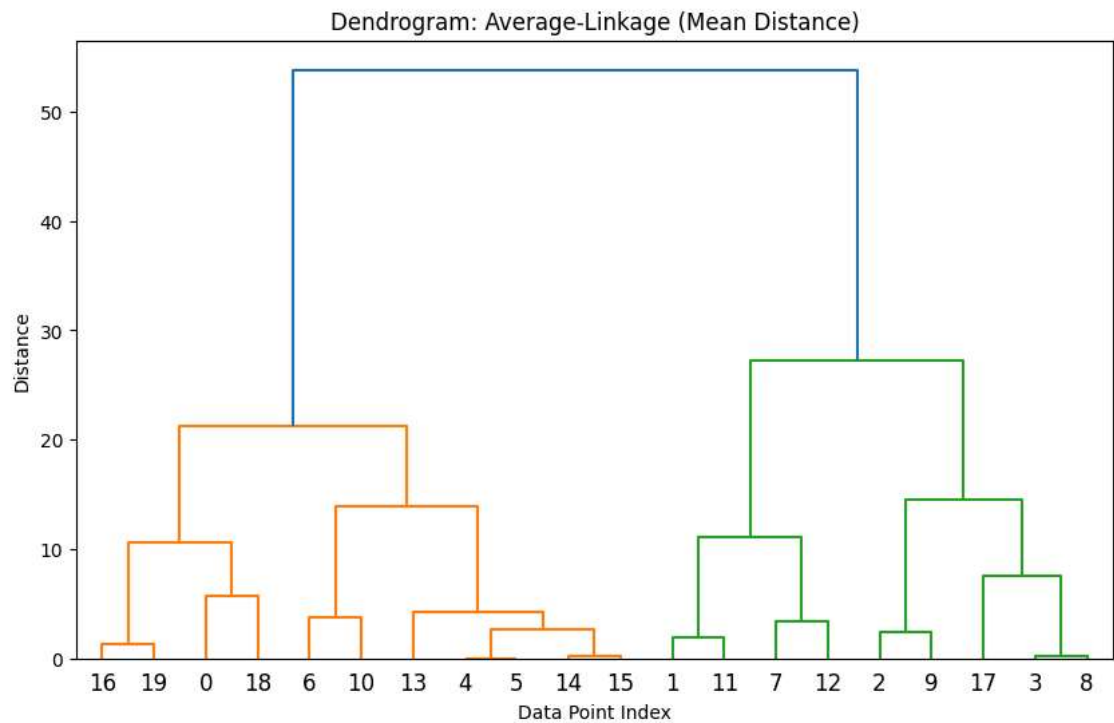
User uploaded file "A3_P2_170cd1d8-a072-4f67-ba36-10f26f772880.csv" with length 379 bytes

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.cluster.hierarchy import dendrogram
5 df = pd.read_csv('A3_P2_170cd1d8-a072-4f67-ba36-10f26f772880.csv')
6 X = df.values
7
8 def euclidean_distance(u, v):
9     return np.sqrt(np.sum((u - v)**2))
10
11 def compute_linkage(cluster_A, cluster_B, X, method):
12     distances = []
13     for i in cluster_A:
14         for j in cluster_B:
15             distances.append(euclidean_distance(X[i], X[j]))
16
17     if method == 'single':
18         return np.min(distances)
19     elif method == 'average':
20         return np.mean(distances)
21     elif method == 'complete':
22         return np.max(distances)
23
24 def hierarchical_clustering(X, method):
25     n = len(X)
26
27     active_clusters = {i: [i] for i in range(n)}
28
29     Z = np.zeros((n - 1, 4))
30     next_cluster_id = n
31
32     for step in range(n - 1):
33         min_dist = float('inf')
34         merge_pair = (-1, -1)
35
36         cluster_ids = list(active_clusters.keys())
37         for i in range(len(cluster_ids)):
38             for j in range(i + 1, len(cluster_ids)):
39                 id_A = cluster_ids[i]
40                 id_B = cluster_ids[j]
41
42                 dist = compute_linkage(active_clusters[id_A], active_clusters[id_B], X, method)
43
44                 if dist < min_dist:
45                     min_dist = dist
46                     merge_pair = (id_A, id_B)
47
48
49         id_A, id_B = merge_pair
50
51         Z[step, 0] = id_A
52         Z[step, 1] = id_B
53         Z[step, 2] = min_dist
54         Z[step, 3] = len(active_clusters[id_A]) + len(active_clusters[id_B])
55
56         active_clusters[next_cluster_id] = active_clusters[id_A] + active_clusters[id_B]
57
58         del active_clusters[id_A]
59         del active_clusters[id_B]
60
61         next_cluster_id += 1
62
63     return Z
64
65 methods = ['single', 'average', 'complete']
66 titles = [
67     'Single-Linkage (Min Distance)',
68     'Average-Linkage (Mean Distance)',
69     'Complete-Linkage (Max Distance)'
70 ]
71
72 for idx, method in enumerate(methods):
73     print(f"Computing {method}-linkage clustering")
74     Z = hierarchical_clustering(X, method)
75
76     plt.figure(figsize=(10, 6))
77     dendrogram(Z)
78     plt.title(f'Dendrogram: {titles[idx]}')
79     plt.xlabel('Data Point Index')
80     plt.ylabel('Distance')
81     plt.show()
```

Computing single-linkage clustering



Computing average-linkage clustering



Computing complete-linkage clustering

